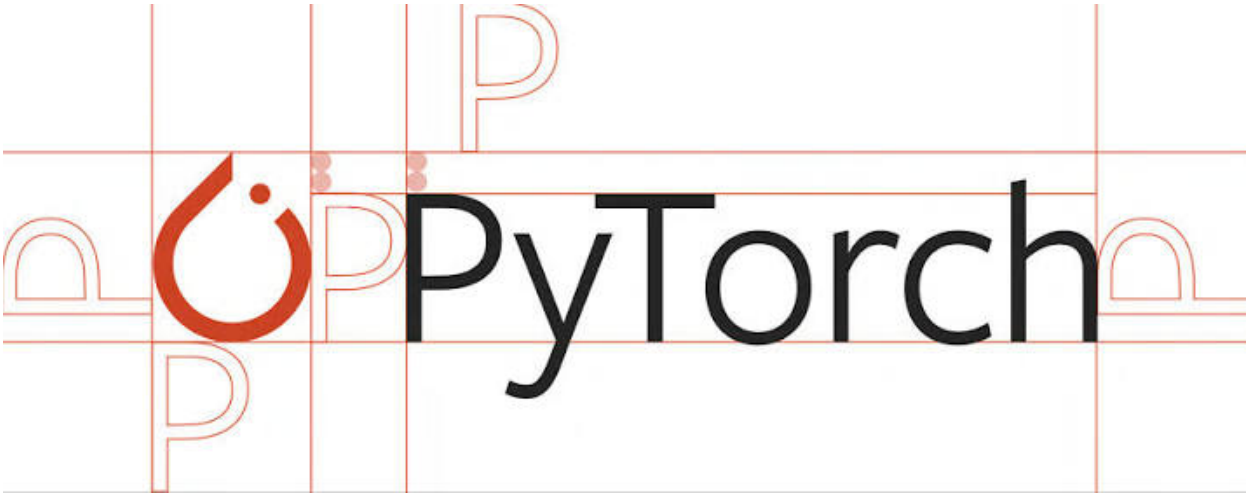


Introduction :-

PyTorch is an open-source machine learning and deep learning framework developed by Facebook's AI Research (FAIR) team. It provides a flexible and intuitive platform for building, training, and deploying neural networks and deep learning models.

PyTorch is widely used by researchers, students, and developers because of its simple Python-like syntax, dynamic computation graph, and strong GPU acceleration. It allows users to create complex deep learning models with less code, making experimentation and debugging much easier.

Today, PyTorch powers many modern AI applications, including image classification, object detection, natural language processing (NLP), speech recognition, recommendation systems, and generative AI models.

History :-

PyTorch was introduced in **2016** by **Facebook's AI Research (FAIR)** laboratory. It was developed by **Adam Paszke**, **Sam Gross**, **Soumith Chintala**, **Gregory Chanan**, **Edward Yang**, and other contributors to provide a flexible and user-friendly deep learning framework.

PyTorch was built on top of the **Torch** scientific computing library, which was written in the **Lua** programming language. Although Torch was powerful, many developers found Lua less familiar compared to Python. To solve this problem, Facebook created PyTorch, combining the power of Torch with Python's simplicity and popularity.

One of PyTorch's biggest innovations was its **dynamic computation graph**, which allows models to be built and modified during execution. This made debugging and

experimentation much easier than with many existing deep learning frameworks at that time.

Over the years, PyTorch has become one of the most widely used deep learning frameworks in both academia and industry. It is now maintained by the **PyTorch Foundation** under the **Linux Foundation** and continues to evolve with regular updates, improved performance, and support for modern AI technologies.

Why PyTorch was Developed :-

PyTorch was developed to provide a **simple, flexible, and efficient** deep learning framework that makes building and training neural networks easier. Before PyTorch, many deep learning frameworks used **static computation graphs**, which made model development and debugging more complex.

The developers wanted a framework that combined the **power of GPU-accelerated deep learning** with the **simplicity of Python programming**. PyTorch introduced a **dynamic computation graph**, allowing users to modify models during runtime, making experimentation, debugging, and research much more convenient.

- To simplify deep learning model development.
- To provide a dynamic computation graph for easier debugging.
- To combine the power of Torch with Python's simplicity.
- To support fast computation using GPUs.
- To make research and experimentation more flexible.
- To bridge the gap between research and production-ready AI models.

Features :-

PyTorch offers a wide range of features that make it one of the most popular frameworks for deep learning and artificial intelligence.

Key Features of PyTorch :-

- **Open Source:** Completely free to use and supported by a large developer community.
- **Python Friendly:** Uses simple and intuitive Python syntax, making it easy to learn.
- **Dynamic Computation Graph:** Models can be modified during runtime, simplifying debugging and experimentation.
- **Automatic Differentiation (Autograd):** Automatically computes gradients required for training neural networks.

- **GPU Acceleration:** Supports CUDA-enabled GPUs for faster model training and inference.
- **Rich Neural Network Library:** Provides the `torch.nn` module to build deep learning models easily.
- **Easy Model Saving and Loading:** Models can be saved and reused using built-in functions.
- **Cross-Platform Support:** Works on Windows, Linux, and macOS.

Applications of PyTorch :-

PyTorch is widely used in the field of Artificial Intelligence (AI) and Deep Learning because of its flexibility, ease of use, and high performance. It enables researchers and developers to build intelligent systems capable of learning from data and making accurate predictions.

- **Computer Vision:** Used for image classification, object detection, image segmentation, and facial recognition.
- **Natural Language Processing (NLP):** Helps build applications such as machine translation, text summarization, sentiment analysis, and chatbots.
- **Speech Recognition:** Used to convert spoken language into text and develop voice assistants.
- **Recommendation Systems:** Powers personalized recommendations for e-commerce platforms, streaming services, and social media.
- **Generative AI:** Used to develop Generative Adversarial Networks (GANs), image generation models, and large language models (LLMs).
- **Healthcare:** Assists in medical image analysis, disease prediction, and drug discovery.
- **Autonomous Vehicles:** Supports object detection, lane detection, and decision-making systems in self-driving cars.
- **Robotics:** Enables robots to learn tasks, recognize objects, and interact intelligently with their environment.

Installation & Import :-

if we are working on your local machine, we can install it using the `pip` package manager.

After installation, the PyTorch library can be imported into your Python program using the `torch` module.

Install PyTorch (For Local Machine) :-

We use the following command to install PyTorch along with the `torchvision` and `torchaudio` libraries.

```
pip install torch torchvision torchaudio
```

Import PyTorch :-

After installation, import the PyTorch library using the `torch` module.

```
import torch
```

Check PyTorch Version :-

We can verify the installed version of PyTorch using the `__version__` attribute.

```
print(torch.__version__)
```

```
2.11.0+cpu
```

Check if GPU (CUDA) is Available :-

PyTorch can use a CUDA-enabled GPU for faster computations. We use the following function to check whether CUDA is available on our system.

Expected Output :-

- **True** → CUDA-enabled GPU is available.
- **False** → PyTorch will use the CPU for computations.

```
print(torch.cuda.is_available())
```

```
False
```

torch.tensor() :-

The `torch.tensor()` function is the primary way to create tensors in PyTorch. We can use this function to convert Python data types such as lists, tuples, and NumPy arrays into PyTorch tensors.

A tensor can contain one-dimensional, two-dimensional, or multi-dimensional data. By default, PyTorch automatically determines the appropriate data type based on the

input values. We can also explicitly specify the data type if needed.

Syntax :-

torch.tensor(data, dtype=None)

Parameters :-

- **data:** The input data used to create the tensor.
- **dtype (Optional):** Specifies the data type of the tensor.

Example :-

In this example, we create a one-dimensional tensor containing integer values.

```
import torch

tensor = torch.tensor([10, 20, 30, 40, 50])

print(tensor)
```

```
tensor([10, 20, 30, 40, 50])
```

The output is a one-dimensional PyTorch tensor containing the values provided in the input list.

PyTorch Properties :-

Every tensor in PyTorch has several properties that describe its characteristics, such as its shape, size, data type, and the device on which it is stored. These properties help us understand the structure of a tensor and are useful while building deep learning models.

Common PyTorch Properties :-

- **shape:** Returns the dimensions of the tensor.
- **size():** Returns the size of the tensor.
- **dtype:** Returns the data type of the tensor.
- **device:** Returns the device on which the tensor is stored (CPU or GPU).
- **ndim:** Returns the number of dimensions of the tensor.
- **numel():** Returns the total number of elements in the tensor.

Example :-

In this example, we create a two-dimensional tensor and access its different properties.

```
import torch

tensor = torch.tensor([[1, 2, 3],
                       [4, 5, 6]])

print("Tensor:\n", tensor)
print("Shape:", tensor.shape)
print("Size:", tensor.size())
print("Data Type:", tensor.dtype)
print("Device:", tensor.device)
print("Dimensions:", tensor.ndim)
print("Total Elements:", tensor.numel())
```

```
Tensor:
 tensor([[1, 2, 3],
        [4, 5, 6]])
Shape: torch.Size([2, 3])
Size: torch.Size([2, 3])
Data Type: torch.int64
Device: cpu
Dimensions: 2
Total Elements: 6
```

PyTorch Operations :-

PyTorch provides a variety of operations that allow us to manipulate and perform mathematical computations on tensors. These operations are similar to those available in NumPy but are optimized for deep learning tasks and can take advantage of GPU acceleration.

Tensor operations can be performed between tensors of the same shape or between compatible shapes through broadcasting. They form the foundation for implementing mathematical calculations, neural network computations, and data transformations in PyTorch.

Common PyTorch Operations :-

- Addition
- Subtraction
- Multiplication
- Division
- Matrix Multiplication

Addition :-

Addition is one of the most common tensor operations in PyTorch. We can add two tensors of the same shape using the `+` operator or the `torch.add()` function.

Example :-

In this example, we add two tensors using the `+` operator.

```
import torch

tensor1 = torch.tensor([10, 20, 30])
tensor2 = torch.tensor([1, 2, 3])

result = tensor1 + tensor2

print("Tensor 1:", tensor1)
print("Tensor 2:", tensor2)
print("Result:", result)
```

```
Tensor 1: tensor([10, 20, 30])
Tensor 2: tensor([1, 2, 3])
Result: tensor([11, 22, 33])
```

Subtraction :-

Subtraction is used to find the difference between two tensors. We can subtract tensors using the `-` operator or the `torch.sub()` function.

Example :-

In this example, we subtract one tensor from another.

```
import torch

tensor1 = torch.tensor([10, 20, 30])
tensor2 = torch.tensor([1, 2, 3])

result = tensor1 - tensor2

print("Tensor 1:", tensor1)
print("Tensor 2:", tensor2)
print("Result:", result)
```

```
Tensor 1: tensor([10, 20, 30])
Tensor 2: tensor([1, 2, 3])
Result: tensor([ 9, 18, 27])
```

Multiplication :-

Multiplication multiplies the corresponding elements of two tensors. We can perform element-wise multiplication using the `*` operator or the `torch.mul()` function.

Example :-

In this example, we perform element-wise multiplication of two tensors.

```
import torch

tensor1 = torch.tensor([10, 20, 30])
tensor2 = torch.tensor([2, 3, 4])

result = tensor1 * tensor2

print("Tensor 1:", tensor1)
print("Tensor 2:", tensor2)
print("Result:", result)
```

```
Tensor 1: tensor([10, 20, 30])
Tensor 2: tensor([2, 3, 4])
Result: tensor([ 20,  60, 120])
```

Matrix Multiplication :-

Matrix multiplication is different from element-wise multiplication. In matrix multiplication, the rows of the first matrix are multiplied by the columns of the second matrix. We can perform matrix multiplication using the `torch.matmul()` function or the `@` operator.

For matrix multiplication, the number of columns in the first matrix must be equal to the number of rows in the second matrix.

Example :-

In this example, we perform matrix multiplication between two matrices.

```
import torch

matrix1 = torch.tensor([[1, 2],
                        [3, 4]])

matrix2 = torch.tensor([[5, 6],
                        [7, 8]])

result = torch.matmul(matrix1, matrix2)

print("Matrix 1:\n", matrix1)
print("Matrix 2:\n", matrix2)
print("Result:\n", result)
```

```
Matrix 1:
 tensor([[1, 2],
        [3, 4]])
Matrix 2:
 tensor([[5, 6],
        [7, 8]])
```

```
Result:
 tensor([[19, 22],
        [43, 50]])
```

Reshaping :-

Reshaping is the process of changing the dimensions of a tensor without changing its data. It allows us to organize tensor data into different shapes, making it suitable for various operations such as matrix computations and neural network inputs.

In PyTorch, we can reshape a tensor using the `reshape()` method. The total number of elements must remain the same before and after reshaping.

Syntax :-

```
tensor.reshape(new_shape)
```

Example :-

In this example, we reshape a one-dimensional tensor into a two-dimensional tensor.

```
import torch

tensor = torch.tensor([1, 2, 3, 4, 5, 6])

reshaped_tensor = tensor.reshape(2, 3)

print("Original Tensor:")
print(tensor)

print("\nReshaped Tensor:")
print(reshaped_tensor)
```

```
Original Tensor:
tensor([1, 2, 3, 4, 5, 6])

Reshaped Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Neural Network Basics :-

A **Neural Network** is a machine learning model inspired by the structure and working of the human brain. It consists of interconnected layers of neurons that learn patterns from data and make predictions.

In PyTorch, neural networks are built using the **torch.nn** module, which provides ready-to-use classes and functions for creating, training, and evaluating deep

learning models.

A basic neural network generally consists of three types of layers:

- **Input Layer:** Receives the input data.
- **Hidden Layer(s):** Processes the input by learning patterns and relationships.
- **Output Layer:** Produces the final prediction or result.

During training, the neural network adjusts its parameters (weights and biases) to minimize the prediction error. This process enables the model to improve its performance over time.

Advantages of Neural Networks :-

- Can learn complex patterns from data.
- Used for both classification and regression tasks.
- Supports applications such as image recognition, speech recognition, and natural language processing.
- Can improve accuracy with sufficient training data.

In the following sections, we will learn how to build a simple neural network using PyTorch.

nn.Module :-

`nn.Module` is the base class for all neural network models in PyTorch. Every custom neural network we create should inherit from this class. It provides the necessary functionality for managing layers, parameters, and forward computations.

When creating a custom neural network, we generally perform the following steps:

1. Inherit the `nn.Module` class.
2. Initialize the layers inside the `__init__()` method.
3. Define how the input data flows through the network using the `forward()` method.

Example :-

In this example, we create a simple neural network class by inheriting from `nn.Module`.

```
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        def forward(self, x):
            return x

    model = SimpleNN()

    print(model)
```

```
    SimpleNN()
```

nn.Linear :-

`nn.Linear` is one of the most commonly used layers in PyTorch. It creates a **fully connected (dense) layer**, where every neuron in the input is connected to every neuron in the output.

This layer performs a linear transformation on the input data by multiplying it with weights and adding a bias.

It is widely used in neural networks for tasks such as classification and regression.

Syntax :-

```
nn.Linear(in_features, out_features)
```

Parameters :-

- **in_features:** Number of input features.
- **out_features:** Number of output features.

Example :-

In this example, we create a neural network with one fully connected (`Linear`) layer.

```
    import torch
    import torch.nn as nn

    class SimpleNN(nn.Module):
        def __init__(self):
            super().__init__()
            self.fc = nn.Linear(3, 2)

        def forward(self, x):
            return self.fc(x)

    model = SimpleNN()

    print(model)
```

```
    SimpleNN(
      (fc): Linear(in_features=3, out_features=2, bias=True)
    )
```

Model Training :-

Model training is the process of teaching a neural network to learn patterns from data. During training, the model makes predictions, compares them with the actual values, calculates the error (loss), and updates its parameters to improve future predictions.

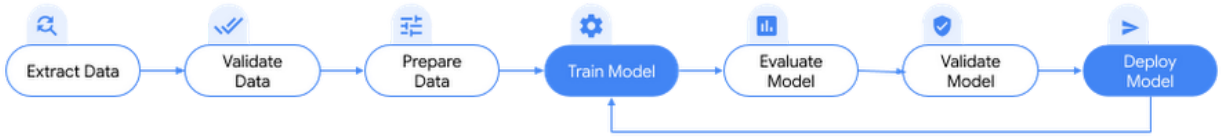
The training process generally consists of the following steps:

1. Pass the input data to the model.
2. Calculate the loss using a loss function.
3. Compute the gradients using backpropagation.
4. Update the model parameters using an optimizer.
5. Repeat these steps for multiple epochs until the model learns effectively.

The two most important components of model training are:

- **Loss Function:** Measures how far the model's predictions are from the actual values.
- **Optimizer:** Updates the model's parameters to minimize the loss.

In the following sections, we will learn about the Loss Function, Optimizer, and the complete Training Loop.



Loss Function :-

A **Loss Function** measures how well a neural network performs by comparing the model's predictions with the actual target values. The goal of training is to minimize this loss so that the model makes more accurate predictions.

PyTorch provides several built-in loss functions through the `torch.nn` module. The choice of loss function depends on the type of machine learning problem.

Some commonly used loss functions are:

- **nn.MSELoss()** – Used for regression problems.
- **nn.CrossEntropyLoss()** – Used for multi-class classification.
- **nn.BCELoss()** – Used for binary classification.

Example :-

In this example, we use `nn.MSELoss()` to calculate the Mean Squared Error (MSE) loss between the predicted values and the target values.

```
import torch
import torch.nn as nn

predictions = torch.tensor([2.5, 0.0, 2.1, 8.0])
targets = torch.tensor([3.0, -0.5, 2.0, 7.0])

loss_function = nn.MSELoss()

loss = loss_function(predictions, targets)

print("Loss:", loss)
```

```
Loss: tensor(0.3775)
```

Optimizer :-

An **Optimizer** is responsible for updating the weights and biases of a neural network to reduce the loss during training. After the loss is calculated, the optimizer adjusts the model's parameters using the computed gradients, helping the model learn from the training data.

PyTorch provides several built-in optimizers in the `torch.optim` module. The choice of optimizer depends on the problem and the desired training performance.

Some commonly used optimizers are:

- **SGD (Stochastic Gradient Descent)**
- **Adam**
- **AdamW**
- **RMSprop**

Syntax :-

```
torch.optim.Adam(model.parameters(), lr=0.001)
```

Parameters :-

- **model.parameters():** Returns all trainable parameters of the model.
- **lr:** Learning rate that controls the size of each parameter update.

Example :-

In this example, we create an Adam optimizer for a simple neural network.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(3, 1)

optimizer = optim.Adam(model.parameters(), lr=0.001)

print(optimizer)
```

```
Adam (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    decoupled_weight_decay: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.001
    maximize: False
    weight_decay: 0
)
```

Model Evaluation :-

Model evaluation is the process of measuring how well a trained model performs on new or unseen data. After training, we evaluate the model to determine whether it has learned meaningful patterns and can make accurate predictions.

Steps for Model Evaluation :-

1. Switch the model to evaluation mode using `model.eval()`.
2. Disable gradient calculations using `torch.no_grad()`.
3. Pass the input data to the model.
4. Compare the predicted values with the actual values.

Example :-

In this example, we evaluate a trained model using sample input data.

```
import torch
import torch.nn as nn

# Sample trained model
model = nn.Linear(1, 1)

# Switch to evaluation mode
model.eval()
```

```
# Sample input
x = torch.tensor([[5.0], [6.0]])

# Disable gradient calculation
with torch.no_grad():
    predictions = model(x)

print(predictions)
```

```
tensor([[-0.0615],
        [-0.2341]])
```

Predictions :-

After training and evaluating a model, the next step is to use it for making **predictions**. Prediction is the process of providing new input data to a trained model and obtaining the corresponding output.

Steps to Make Predictions :-

1. Switch the model to evaluation mode using `model.eval()`.
2. Disable gradient calculations using `torch.no_grad()`.
3. Pass the new input data to the model.
4. Obtain the predicted output.

Example :-

In this example, we use a trained model to make predictions for new input values.

```
import torch
import torch.nn as nn

# Sample model
model = nn.Linear(1, 1)

# Switch to evaluation mode
model.eval()

# New input data
new_data = torch.tensor([[7.0], [8.0]])

# Make predictions
with torch.no_grad():
    predictions = model(new_data)

print("Predicted Output:")
print(predictions)
```

```
Predicted Output:
tensor([[6.4194],
        [7.3852]])
```

Saving & Loading Models :-

After training a neural network, we often want to save it so that we can use it later without training it again. PyTorch provides simple functions to save and load models, making it easy to reuse trained models, share them with others, or deploy them in real-world applications.

PyTorch provides two main functions for this purpose:

- **torch.save()** - Saves a model or its parameters to a file.
- **torch.load()** - Loads the saved model or its parameters from a file.

In the following sections, we will learn how to save and load models using these functions.

torch.save() :-

After training a model, it is often useful to save it so that we can use it later without training it again. In PyTorch, the `torch.save()` function is used to save a model or its parameters to a file.

Syntax :-

```
torch.save(model.state_dict(), "model.pth")
```

Parameters :-

- **model.state_dict():** Contains all the learnable parameters (weights and biases) of the model.
- **"model.pth":** The name of the file in which the model will be saved.

Example :-

In this example, we create a simple model and save its parameters to a file.

```
import torch
import torch.nn as nn

# Create a simple model
model = nn.Linear(2, 1)

# Save the model parameters
```

```
torch.save(model.state_dict(), "model.pth")

print("Model saved successfully!")
```

```
Model saved successfully!
```

torch.load() :-

The `torch.load()` function is used to load a previously saved model or its parameters from a file. It allows us to restore the learned weights and biases without retraining the model.

Syntax :-

```
model.load_state_dict(torch.load("model.pth"))
```

Parameters :-

- **torch.load("model.pth"):** Loads the saved model parameters from the file.
- **load_state_dict():** Copies the loaded parameters into the model.

Example :-

In this example, we load the saved parameters into a new model.

```
import torch
import torch.nn as nn

# Create the same model architecture
model = nn.Linear(2, 1)

# Load the saved model parameters
model.load_state_dict(torch.load("model.pth"))

# Set the model to evaluation mode
model.eval()

print("Model loaded successfully!")
```

```
Model loaded successfully!
```

Conclusion :-

PyTorch is one of the most popular and powerful deep learning frameworks used for building machine learning and artificial intelligence applications. Its simple syntax, dynamic computation graph, and strong GPU support make it suitable for both beginners and experienced developers.

In this notebook, we explored the fundamental concepts of PyTorch, including tensors, tensor operations, neural networks, model training, evaluation, predictions, and saving and loading models. These concepts form the foundation for developing deep learning models using PyTorch.

Key Takeaways :-

- PyTorch uses tensors as its primary data structure.
- Tensor operations are efficient and support GPU acceleration.
- Neural networks are built using the `torch.nn` module.
- Models are trained using loss functions and optimizers.
- Evaluation and prediction help measure a model's performance.
- Trained models can be saved and loaded using `torch.save()` and `torch.load()`.